H2S-NN: Heterogeneous Heterophilic Sheaf Neural Network with Path Instances

Anonymous submission

Abstract

In a heterogeneous graph, nodes and relations have several types; under heterophily, connected nodes often differ in label or features, so averaging neighbours is unreliable. Meta-paths such as paper–author–paper describe how two nodes are related through intermediate nodes. Existing methods use them either by splitting the graph into one homogeneous graph per meta-path, which removes the intermediate nodes from the computation, or by encoding each path instance into a feature of its target node only. We ask whether path instances can instead act inside the original graph. We propose the Heterogeneous Heterophilic Sheaf Neural Network with Path Instances (H2S-NN), a sheaf neural network on the original heterogeneous graph: each edge has learned maps that transform a message between its endpoints, so neighbours need not agree directly. Path instances are sampled by random walks without a meta-path list; each receives a weight computed from its own nodes and relation sequence, and every node of the instance, including intermediate nodes, receives its message through the same edge maps the graph uses. Edge and instance terms form one normalized operator, stable for any parameter values, to which a learned spectral filter is applied. On node-classification benchmarks spanning a range of heterophily, [result]; ablations on identical sampled instances show [result].

1 Introduction

Heterogeneous graphs are a common representation for modeling complex relational structures, such as citation networks [Liu *et al.*, 2014] or social interactions [Singh and Verma, 2025]. Unlike homogeneous graphs, heterogeneous graphs contain multiple types of nodes and relations with distinct semantic roles [Sun and Han, 2013]. Moreover, many graphs are also heterophilic, meaning that adjacent nodes may have different labels or attributes, which makes similarity-based neighbor aggregation unreliable. A graph can be both heterogeneous and heterophilic [Lin *et al.*, 2025]; in Figure 1(a), papers P1 and P9 share the author A7 but belong to different fields. Modeling such graphs under the simplifying assumptions of homogeneity or homophily can obscure the relational patterns that distinguish their nodes and consequently degrade predictive performance [Lin *et al.*, 2025].

Meta-paths are the standard way to express typed multi-hop relations. A meta-path schema such as `Paper-Author-Paper` (PAP) describes a co-authorship relation that is contained in neither endpoint alone [Sun *et al.*, 2011], and a path instance such as P1–A7–P9 is one concrete walk that follows it. This raises the question that motivates this work: *P1 and P9 are linked through the author A7 and through the subject S3; does it matter who A7 and S3 are, or is it enough to know that P1 and P9 are connected?*

Existing methods answer this question in different ways. Most methods for heterogeneous heterophilic graphs split the graph by meta-path schema into latent or induced homogeneous graphs [Lu *et al.*, 2026; Chen and Song, 2025; Li *et al.*, 2025; Guo *et al.*, 2023; Shen and Kang, 2025; Wang *et al.*, 2025], or precompute neighbour averages per schema [Yang *et al.*, 2023; Li *et al.*, 2024]. As Figure 1(b) shows, the intermediate nodes A7 and S3 then no longer take part in the computation. Instance encoders such as MAGNN [Fu *et al.*, 2020], MECCH [Fu and King, 2024], and MF2Vec [Kim *et al.*, 2024] do keep the intermediate nodes: they encode each instance and aggregate the result into its target node. However, the encoding is written only to the target, and each schema is processed in its own subgraph, so A7 itself never learns about the pair (P1, P9). Sheaf neural networks handle heterophily by comparing neighbours only after a learned transformation [Hansen and Gebhart, 2020; Bodnar *et al.*, 2022], and HetSheaf applies them to the original heterogeneous graph [Braithwaite *et al.*, 2024], but only edges enter its operator. Finally, a recent study shows that a tuned R-GCN matches or outperforms many heterogeneous graph neural networks [Yang *et al.*, 2025], so a new component must be justified by controlled comparisons rather than by benchmark rankings alone.

To address this gap, we introduce the **Heterogeneous Heterophilic Sheaf Neural Network with Path Instances (H2S-NN)**. H2S-NN works on the original graph. Each edge carries learned maps that transform a message from one endpoint into the space of the other, so neighbours need not agree directly. Path instances are sampled by random walks; each adds one term to the same operator, in which the nodes of the instance are compared after transport along the path,

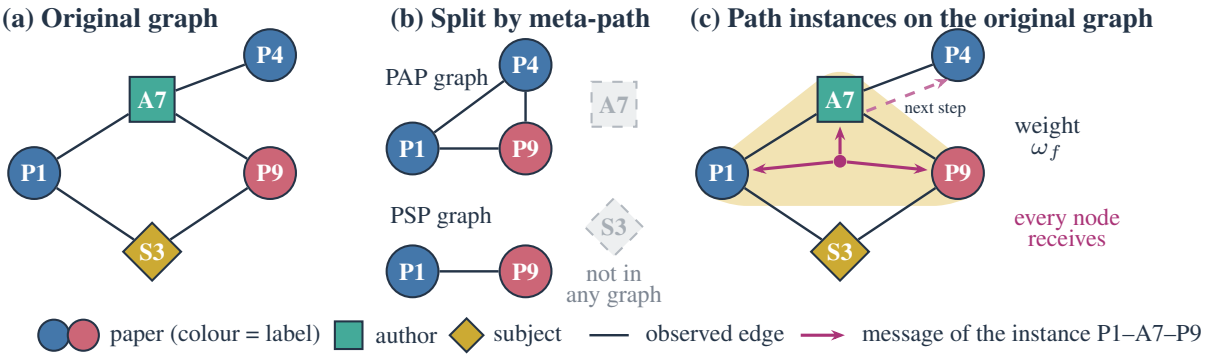


Figure 1: Running example. **(a)** A heterogeneous heterophilic citation graph: papers P1, P4, P9 (colour = label), author A7, subject S3. **(b)** Splitting by meta-path builds one paper graph per schema (PAP, PSP); A7 and S3 appear in neither. **(c)** H2S-NN keeps the path instance P1–A7–P9 on the original graph: its message (arrows) reaches all three of its nodes, scaled by the instance weight ω_f, and A7 passes it to P4 at the next step. The shaded region only marks the nodes of the instance.

and every node of the instance, not only its end node, receives the correction (Figure 1(c)). Our main contributions are summarized as follows:

1. We let path instances act inside the propagation operator of the original heterogeneous graph: each instance is weighted individually from its own nodes and relations, and every node of the instance, including intermediate nodes, receives the instance's message, transformed by the same edge maps the graph uses.
2. We develop Walk-sampled, Instance-weighted Sheaf Paths (WISP), a sampler that draws path instances by random walks from a stated distribution with unbiased weights, so the model needs no meta-path list and no per-meta-path parameters, and its cost does not grow with the number of schemas.
3. We design an evaluation that isolates each design choice on identical sampled instances: no instances, endpoint-only weights, free path maps, endpoint shortcut edges, and a naive combination of MAGNN-style instance features with an edge-only sheaf model; known-answer tasks are reported, including the one that failed.

2 Preliminaries

2.1 Heterogeneous Graphs

A heterogeneous graph is a tuple $G = (V, E, \phi, \psi)$, where V and E are the node and edge sets, $\phi : V \to \mathcal{T}$ assigns each node a type, and $\psi : E \to \mathcal{R}$ assigns each edge a relation type. The graph is heterogeneous when $|\mathcal{T}| > 1$ or $|\mathcal{R}| > 1$; it reduces to a homogeneous graph when $|\mathcal{T}| = |\mathcal{R}| = 1$ [Lu *et al.*, 2026]. Benchmarks often store each fact twice, once per direction; we keep one oriented edge per fact.

2.2 Meta-Path Schemas and Path Instances

A meta-path schema of length ℓ is a typed relation sequence

$$\pi : \; t_0 \xrightarrow{r_1} t_1 \xrightarrow{r_2} \cdots \xrightarrow{r_\ell} t_\ell, \qquad t_i \in \mathcal{T},\; r_i \in \mathcal{R}, \tag{1}$$

which specifies a composite relation between its endpoint types [Sun *et al.*, 2011]. A path instance is an ordered sequence $f = (v_0, e_1, v_1, \ldots, e_\ell, v_\ell)$ of nodes and edges with $\phi(v_i) = t_i$ and $\psi(e_i) = r_i$. Thus, π describes a type-level pattern, whereas f identifies one occurrence of that pattern in the observed graph; we call $v_1, \ldots, v_{\ell-1}$ the intermediate nodes of f. For example, PAP is a schema and P1–A7–P9 is one of its instances, with A7 as its intermediate node.

2.3 Homophily and Heterophily

Homophily refers to the prevalence of edges between nodes that share the same label. In a heterogeneous graph, labels are usually comparable only between nodes of the target type, so heterophily is measured on the graphs induced by meta-paths. Let $\mathcal{P}_2$ be the set of length-two schemas from the target type to itself and G_p the graph that $p \in \mathcal{P}_2$ induces on target nodes. We report two measures exactly as their sources define and implement them. The primary measure is the $\mathcal{H}^2$ index of H2GB [Lin *et al.*, 2025],

$$\mathcal{H}^2(G) = \frac{1}{|\mathcal{P}_2|} \sum_{p \in \mathcal{P}_2} \big(1 - h_{\mathrm{adj}}(G_p)\big), \tag{2}$$

where h_{adj} is the adjusted homophily of Platonov et al. [2023], which corrects edge homophily for class imbalance. Values near 0 indicate homophily, and values near or above 1 indicate heterophily. Next to it we report the metapath-based label heterophily (MLH) [Li *et al.*, 2025], which averages the uncorrected edge heterophily over the same graphs.

2.4 Cellular Sheaves on Graphs

A cellular sheaf $\mathcal{F}$ on a graph assigns a vector space, or *stalk*, to every node and edge, and a linear restriction map $F_{v \trianglelefteq e} : \mathcal{F}(v) \to \mathcal{F}(e)$ to every incident pair. We call these maps *edge maps*: $F_{v \trianglelefteq e}$ expresses the state of node v in the space of edge e. With d-dimensional stalks, the coboundary compares the two endpoints of an oriented edge $e = (u \to v)$ after both are mapped into the edge stalk:

$$(\delta x)_e = F_{v \trianglelefteq e} x_v - F_{u \trianglelefteq e} x_u. \tag{3}$$

The sheaf Laplacian $\Delta_0 = \delta^{\mathsf{T}} \delta$ has energy $x^{\mathsf{T}} \Delta_0 x = \|\delta x\|^2$; when all maps are identities, it recovers the graph Laplacian. Sheaf neural networks can address heterophily and

oversmoothing by diffusing through learned edge maps instead of averaging raw features [Hansen and Gebhart, 2020; Bodnar *et al.*, 2022], which has led to numerous extensions [Barbero *et al.*, 2022; Fiorini *et al.*, 2025; Ribeiro *et al.*, 2026], including HetSheaf for heterogeneous graphs [Braithwaite *et al.*, 2024]. These models define the sheaf only over node–edge incidences.

3 Methodology

3.1 Problem Formulation

We consider supervised prediction on a heterogeneous graph $G = (V, E, \phi, \psi, X^{\mathrm{raw}})$ in which neighboring nodes may have different labels or features. Here, $X^{\mathrm{raw}} = \{x_v^{\mathrm{raw}}\}_{v \in V}$ contains type-dependent node features with $x_v^{\mathrm{raw}} \in \mathbb{R}^{d_{\phi(v)}}$; node types without features receive a learned vector per node. Unlike meta-path methods, the model is given no list of meta-path schemas: path instances are drawn by the model itself (Section 3.5). The learning problem is to construct a predictor $\hat{y}_o = g_\theta(o; G)$ for a supervised object o, which is a target node for node classification, a typed node pair for link prediction, and a graph for graph classification. Given labeled training objects $\mathcal{S}_{\mathrm{train}}$, the parameters are learned by minimizing

$$\min_\theta \frac{1}{|\mathcal{S}_{\mathrm{train}}|} \sum_{o \in \mathcal{S}_{\mathrm{train}}} \mathcal{L}\big(g_\theta(o; G), y_o\big), \tag{4}$$

where $\mathcal{L}$ is cross-entropy for single-label targets and binary cross-entropy for multilabel targets and link prediction.

3.2 Overview

As shown in Figure 2, H2S-NN consists of six steps. First, *node encoding* maps typed features into a shared d-dimensional stalk space and adds a summary of each node's neighbours. Second, *edge maps* define a sheaf on the original graph. Third, WISP samples path instances by random walks, and fourth, each instance receives a learned weight. Fifth, every instance adds an *instance term* that compares its nodes after transport along the path. Finally, edge and instance terms form one normalized operator, to which a gated Chebyshev filter bank is applied before a prediction head. The edge maps, gates, and instance weights are computed once per forward pass from the encoded nodes, so all propagation acts on one sheaf.

3.3 Node Encoding and Neighbourhood Summary

H2S-NN first maps type-dependent node features into a shared d-dimensional stalk space, so that nodes of different types can be compared by the same edge maps. For each node v, we compute

$$h_v^{(0)} = \mathrm{GELU}\big(\mathrm{LN}\big(W_{\phi(v)} x_v^{\mathrm{raw}} + b_{\phi(v)}\big) + q_{\phi(v)}\big) \in \mathbb{R}^d, \tag{5}$$

where W_t and b_t are type-specific parameters and q_t is a learned type embedding. Two nodes with similar features can still play different relational roles, for example a prolific and a junior author. We therefore add a parameter-free summary of each node's typed neighbourhood: for every relation type r and role $\rho \in \{\mathrm{src}, \mathrm{tgt}\}$, c_v concatenates the mean of $h^{(0)}$ over the neighbours of v under (r, ρ). The edge maps and gates below read $[h_v^{(0)}, c_v]$; they are computed once, so the geometry does not change during propagation.

3.4 Edge Maps: a Sheaf on the Original Graph

Each edge transforms messages between its endpoints, so that heterophilic neighbours are compared after a learned change of coordinates rather than directly. For an oriented edge $e = (u \xrightarrow{r} v)$, the edge maps are

$$F_{u \trianglelefteq e} = \lambda_e R_{r,\mathrm{src}} H_{u,e}, \qquad F_{v \trianglelefteq e} = \lambda_e R_{r,\mathrm{tgt}} H_{v,e}, \tag{6}$$

where $R_{r,\rho} \in SO(d)$ is a learned frame for relation r and role ρ, $H_{z,e}$ is a product of k Householder reflections whose vectors are predicted from $[h_u^{(0)}, c_u]$, $[h_v^{(0)}, c_v]$, the relation embedding ζ_r and the role of z, and $\lambda_e > 0$ is one scale per edge. Thus, the maps depend on the relation, on the direction of the edge and on both endpoints, and every map is a positive multiple of an orthogonal matrix. The node operator over edges is the sheaf Laplacian Δ_0 of Equation (3). Because the two maps of an edge share one scale, the scale cancels when a message crosses the edge, and every composition of edge maps along a path is orthogonal.

3.5 WISP: Schema-Free Sampling of Path Instances

WISP answers which path instances can enter the model without a meta-path list. From every node of the target type (the *anchor*), it draws r random walks on the original graph. A walk first draws its length ℓ uniformly from $\{2, \ldots, \ell_{\max}\}$; at each hop it chooses uniformly among the available relation–direction pairs that still lead to an unvisited node, and then chooses uniformly among those neighbours. A walk that reaches a dead end before its length is discarded. This defines a probability $p(\omega \mid a)$ for every simple walk ω from anchor a, and the operator that the instances are meant to represent,

$$L_P^{\mathrm{target}} = \sum_a \sum_\omega p(\omega \mid a)\, L_{f(\omega)}, \tag{7}$$

where $f(\omega)$ is the instance traced by ω and L_f is its instance term (Section 3.7). Walks may be drawn from a mixture $\frac{1}{2}p + \frac{1}{2}q$ with any proposal q, and each instance receives the importance weight

$$m_f = \sum_{\omega \to f} \frac{1}{r} \frac{p(\omega)}{\frac{1}{2}p(\omega) + \frac{1}{2}q(\omega)}, \tag{8}$$

summed over the sampled walks that trace f; a path and its reversal are one instance. Each walk contributes at most $2/r$, whatever its length. With these weights, $\mathbb{E}[\sum_f m_f L_f] = L_P^{\mathrm{target}}$ for every proposal (Appendix A); our experiments use $q = p$. Training draws new walks every epoch, and evaluation uses one fixed draw, so every compared model sees the same evaluation instances. The schema of an instance (e.g., PAP) is a label read from the walk, not an input. Hence, the number of candidate schemas does not enter the cost, and the share of instance weight per schema can be reported after training.

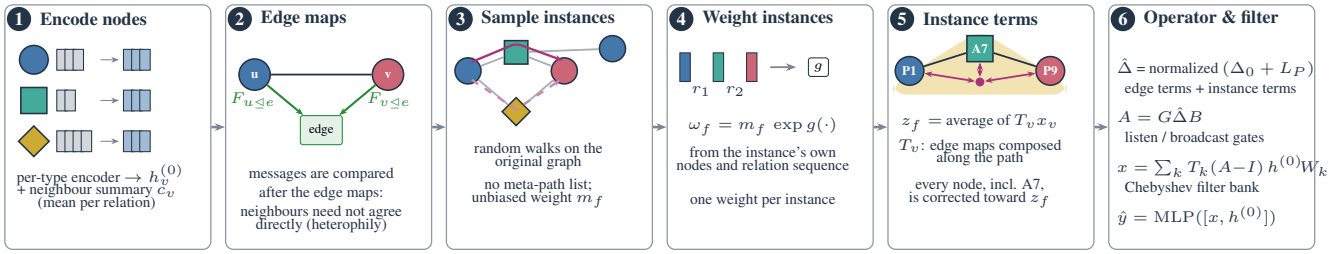


Figure 2: Overview of H2S-NN. **(1)** Type-specific encoders map node features to a common d-dimensional stalk; a parameter-free summary c_v of each node's neighbours is added. **(2)** Each edge has learned maps $F_{u \trianglelefteq e}, F_{v \trianglelefteq e}$; endpoint messages are compared only after these maps. **(3)** WISP samples path instances by random walks on the original graph, without a meta-path list, with an unbiased weight m_f. **(4)** Each instance receives a weight ω_f from its own nodes and relation sequence. **(5)** The nodes of an instance are transported along the path with the edge maps, compared with their average z_f, and all of them are corrected, including the intermediate node A7. **(6)** Edge and instance terms form one normalized operator, filtered with listen/broadcast gates and a Chebyshev filter bank; a head reads $[x, h^{(0)}]$.

3.6 Instance Weights

Instances of the same schema can carry different information: a PAP instance through a prolific, off-topic author may deserve less trust than another PAP instance. Each instance therefore receives its own positive weight,

$$\begin{aligned} \omega_f &= m_f \exp\big(g(\xi_f)\big), \\ \xi_f &= \big[h_{v_0}^{(0)} \,\|\, \cdots \,\|\, h_{v_\ell}^{(0)} \,\|\, \zeta_{r_1} \,\|\, \cdots \,\|\, \zeta_{r_\ell}\big], \end{aligned} \tag{9}$$

where g is a small network for each path length that reads every node of the instance in path order and its relation sequence. The weight corrects the sampling through m_f and scores the instance through g. The input ξ_f uses $h^{(0)}$ and not the neighbourhood summary c, so that a variant whose g reads only the endpoints indeed has no access to the intermediate nodes. Uniform weights, schema-only weights and endpoint-only weights are restrictions of Equation (9), which we use as controls in Section 5.

3.7 Instance Terms

Each instance adds one term to the operator, which asks all of its nodes to agree after transport along the path. For an instance $f = (v_0, e_1, v_1, \ldots, e_\ell, v_\ell)$, the transport brings the stalk of every node of f into the frame of v_0 by composing the edge maps along the path:

$$T_{v_0} = I, \qquad T_{v_i} = T_{v_{i-1}} F_{v_{i-1} \trianglelefteq e_i}^{-1} F_{v_i \trianglelefteq e_i}, \quad i = 1, \ldots, \ell. \tag{10}$$

Here, $F_{v_i \trianglelefteq e_i} x_{v_i}$ expresses x_{v_i} in the space of the edge e_i, the inverse map returns it to the frame of v_{i-1}, and $T_{v_{i-1}}$ continues to v_0. The transports are therefore not free parameters: they are fixed by the edge maps of the instance's own edges. The instance term then compares each transported state with the average of the instance,

$$\begin{aligned} z_f &= \frac{1}{\ell+1} \sum_{v \in f} T_v x_v, \\ x^\top L_P x &= \sum_f \omega_f \sum_{v \in f} \|T_v x_v - z_f\|^2 . \end{aligned} \tag{11}$$

For P1–A7–P9, z_f averages the three transported states, and the gradient of the term moves P1, A7, and P9 toward z_f; hence the intermediate node A7 receives the message of the instance and, through its edge to P4, passes it on at the next application of the operator.

Three properties follow directly. First, an instance never contradicts the edges: if $\delta x = 0$, then $F_{v_{i-1} \trianglelefteq e_i} x_{v_{i-1}} = F_{v_i \trianglelefteq e_i} x_{v_i}$ on every edge of f, so all $T_v x_v$ are equal and the term vanishes; adding instances keeps every state that the edges already accept. Second, computationally an instance term is equivalent to one additional hub node joined to every node of the instance, with the transports T_v as its edge maps and the hub state eliminated, because the minimizer of $\sum_v \|T_v x_v - z\|^2$ over z is z_f (Appendix A, Prop. 2). Third, if the edge maps depended only on the relation type, z_f would coincide with the relational rotation encoder of MAGNN [Fu *et al.*, 2020]. H2S-NN differs in that its maps depend on both endpoints, are the same maps the edges use, and return the message to every node of the instance rather than to the target alone.

3.8 Operator, Gates, and Filter

Edge terms and instance terms form one operator. To keep it stable for any learned parameters, we normalize it by its block-diagonal degree D:

$$\hat{\Delta} = D^{-1/2}\big(\Delta_0 + L_P\big)D^{-1/2}. \tag{12}$$

Because every edge and instance term L satisfies $L \preceq 2D_L$ for its own degree, the spectrum of $\hat{\Delta}$ lies in $[0, 2]$ for every value of the parameters (Appendix A, Prop. 3). Under heterophily, influence need not be symmetric: a subject shared by thousands of papers should inform them without being pulled toward each one. We therefore add a listen gate $\gamma_u \in (0, 1)$ and a broadcast gate $\beta_v \in (0, 1)$ per node, read from $[h^{(0)}, c]$, and use $A = \Gamma\hat{\Delta}B$ with $\Gamma = \mathrm{diag}(\gamma)$ and $B = \mathrm{diag}(\beta)$. The influence of v on u is thus scaled by $\gamma_u\beta_v$: if $\gamma_u = 0$, node u ignores the others, and if $\beta_v = 0$, node v influences no one. A is self-adjoint in the node metric $B\Gamma^{-1}$, so its spectrum is real and lies in $[0, 2)$.

Part of the label signal under heterophily lies in disagreement between transported neighbours, that is, in high and middle frequencies of the operator, which repeated first-order diffusion only damps. We therefore apply a Chebyshev filter

bank [Defferrard *et al.*, 2016; He *et al.*, 2022] with a separate response per stalk coordinate:

$$x = \sum_{k=0}^{K} C_k(A - I)\, h^{(0)} W_k, \qquad \sum_k \|W_k\|_2 \le 1, \quad (13)$$

where C_k is the k-th Chebyshev polynomial and $W_k \in \mathbb{R}^{d\times d}$ mixes stalk coordinates within each frequency band. Since the spectrum of $A - I$ lies in $[-1, 1)$, every $C_k(A - I)$ has norm at most one, and the filter has gain at most one for all parameter values (Appendix A, Prop. 5). Finally, the prediction for node v is $\hat{y}_v = \mathrm{MLP}([x_v \,\|\, h_v^{(0)}])$; keeping the node's own encoding separate from the propagated state avoids mixing a node with disagreeing neighbours before the nonlinear head [Zhu *et al.*, 2020]. Link prediction uses the same states with a pair decoder, and graph classification uses sum pooling (Appendix B).

Complexity. The edge maps are applied as one relation frame and k reflections, so no $d \times d$ matrix is stored per edge. Each application of A costs $O\big((|E| + I)(d^2 + kd)\big)$, where I is the number of node–instance incidences; the cost is linear in edges and incidences and does not depend on the number of schemas.

4 Experiments

4.1 Experiment Settings

To assess H2S-NN across heterogeneous graph learning settings, we evaluated it on node classification, the main task, and on link prediction; graph classification is reported in Appendix C as a check that the model covers the task.

Datasets. For node classification, we used four HGB benchmarks [Lv *et al.*, 2021], ACM, DBLP, IMDB, and Freebase, and four H2GB benchmarks [Lin *et al.*, 2025], IEEE-CIS-G, mag-year, ogbn-mag, and H-Pokec, which together cover a wide range of heterophily (Table 1). We computed $\mathcal{H}^2$ and MLH with the reference implementations; for IMDB, whose movies have several genres, the measures are averaged over the five binary genre labels. For link prediction, we derived tasks from the node-classification graphs, ACM paper–author, DBLP author–paper, IMDB movie–actor, and Freebase book–book, and used HGB PubMed and LastFM [Lv *et al.*, 2021]. In the link-prediction files, node features that are computed from the target relation or describe the other endpoint are removed, because they reveal held-out links. Dataset statistics are given in Appendix C.

Baselines. For node classification, we compared H2S-NN with baselines from four model categories. The homogeneous homophilic baselines comprised GCN [Kipf and Welling, 2017], GAT [Veličković *et al.*, 2018], and ChebNetII [He *et al.*, 2022], while the homogeneous heterophilic baselines included NSD [Bodnar *et al.*, 2022] and CSNN [Ribeiro *et al.*, 2026]. For heterogeneous graphs, we considered the homophilic models R-GCN [Schlichtkrull *et al.*, 2018], HAN [Wang *et al.*, 2019], MAGNN [Fu *et al.*, 2020], SeHGNN [Yang *et al.*, 2023], HiNormer [Mao *et al.*, 2023], and PSHGCN [He *et al.*, 2024], as well as the heterophilic models H2G-former [Lin *et al.*, 2025], Hetero^{2}Net [Li *et al.*, 2025], H2SGNN [Lu *et al.*, 2026], HetSheaf [Braithwaite *et al.*, 2024], and LatGRL [Shen and Kang, 2025]. For link prediction, we used a subset of these baselines while retaining representatives from all four categories.

Evaluation Settings. For node classification, we followed the official splits of HGB (24%/6%/70%) and H2GB and reported Macro-F1 and Micro-F1 (H2GB datasets: the metric of the benchmark). IMDB is evaluated as five binary genre labels with a fixed threshold. For link prediction, the target relation was split by fact into 81%/9%/10%, and every held-out fact was removed in both directions from the message graph and from every sampled instance. We reported MRR and Hits@10 against hard negatives for each positive, built as in HeaRT [Li *et al.*, 2023] with candidates restricted to the endpoint type, separately for pairs whose endpoints are seen in training and for cold-start pairs. All methods, including every baseline, were rerun under one protocol with the same tuning budget, model selection on validation only, and ten seeds; we report means and standard deviations and test each comparison with a paired Wilcoxon test with Holm correction. The WISP settings were fixed before any run and never tuned: $\ell_{\max} = 4$ on HGB, and r chosen so that the number of node–instance incidences matches that of the literature metapaths on the same dataset. Following HGB [Lv *et al.*, 2021], we group the baselines as Hom./hom., Hom./het., Het./hom., and Het./het., where the first term indicates whether the graph is modeled as homogeneous or heterogeneous and the second whether the method is designed for homophilic or heterophilic settings.

4.2 Results

Table 1 reports node classification. [result]

Table 2 reports link prediction. [result]

5 Discussion

5.1 Ablation Study: What the Instances Contribute

To isolate the contribution of each design choice, we compared the full H2S-NN with variants that use the same sampled instances, the same evaluation draw, and parameter counts within 5% (Table 3). *No instances* removes L_P. The *weight ladder* restricts g in Equation (9) to a constant (uniform), to the relation sequence (schema), or to the endpoints and relation sequence (endpoints); the full model differs from the endpoints variant only in seeing the intermediate nodes. *Endpoint shortcut* replaces each instance by one edge between its endpoints carrying the composed transport, which is what eliminating the intermediate nodes leaves (Appendix A) and what a closure edge between the endpoints computes. *Free path maps* replaces the transports by freely learned maps per node and instance, as in sheaf hypergraph networks [Duta *et al.*, 2023]. The *naive combination* encodes the same instances with MAGNN's rotation encoder and instance attention and feeds them as node features to the model without instances, which tests whether joining the two ideas inside one operator adds anything over placing them side by side. The criterion was fixed before running: if the full model is not significantly better than the endpoints variant on at least

Table 1: Node classification (Macro-F1 for HGB; benchmark metric for H2GB), mean ± std over ten seeds. The second header row gives $\mathcal{H}^2$ (higher = more heterophilic). Best values are bold.

Group	Model	ACM [result]	DBLP [result]	IMDB 0.737	Freebase 0.995	IEEE-CIS-G [result]	mag-year [result]	ogbn-mag [result]	H-Pokec [result]
Hom./hom.	GCN	[result]	[result]	[result]	[result]	[result]	[result]	[result]	[result]
	GAT	[result]	[result]	[result]	[result]	[result]	[result]	[result]	[result]
	ChebNetII	[result]	[result]	[result]	[result]	[result]	[result]	[result]	[result]
Hom./het.	NSD	[result]	[result]	[result]	[result]	[result]	[result]	[result]	[result]
	CSNN	[result]	[result]	[result]	[result]	[result]	[result]	[result]	[result]
Het./hom.	R-GCN	[result]	[result]	[result]	[result]	[result]	[result]	[result]	[result]
	HAN	[result]	[result]	[result]	[result]	[result]	[result]	[result]	[result]
	MAGNN	[result]	[result]	[result]	[result]	[result]	[result]	[result]	[result]
	SeHGNN	[result]	[result]	[result]	[result]	[result]	[result]	[result]	[result]
	HiNormer	[result]	[result]	[result]	[result]	[result]	[result]	[result]	[result]
	PSHGCN	[result]	[result]	[result]	[result]	[result]	[result]	[result]	[result]
Het./het.	H2G-former	[result]	[result]	[result]	[result]	[result]	[result]	[result]	[result]
	Hetero^{2}Net	[result]	[result]	[result]	[result]	[result]	[result]	[result]	[result]
	H2SGNN	[result]	[result]	[result]	[result]	[result]	[result]	[result]	[result]
	HetSheaf	[result]	[result]	[result]	[result]	[result]	[result]	[result]	[result]
	LatGRL	[result]	[result]	[result]	[result]	[result]	[result]	[result]	[result]
Ours	H2S-NN	[result]	[result]	[result]	[result]	[result]	[result]	[result]	[result]

Table 2: Link prediction, MRR with HeaRT hard negatives (mean over ten seeds). Hits@10, AUROC/AUPR, and the seen/cold-start split are in Appendix C.

Model	ACM	DBLP	IMDB	Freebase	PubMed	LastFM
GCN	[result]	[result]	[result]	[result]	[result]	[result]
NSD	[result]	[result]	[result]	[result]	[result]	[result]
R-GCN	[result]	[result]	[result]	[result]	[result]	[result]
MAGNN	[result]	[result]	[result]	[result]	[result]	[result]
HetSheaf	[result]	[result]	[result]	[result]	[result]	[result]
H2G-former	[result]	[result]	[result]	[result]	[result]	[result]
H2S-NN	[result]	[result]	[result]	[result]	[result]	[result]

Table 3: Ablations on identical sampled instances (Macro-F1, mean over ten seeds; †: significantly below the full model, paired Wilcoxon with Holm correction).

Variant	ACM	DBLP	IMDB	Freebase
H2S-NN (full)	[result]	[result]	[result]	[result]
No instances	[result]	[result]	[result]	[result]
Uniform weights	[result]	[result]	[result]	[result]
Schema weights	[result]	[result]	[result]	[result]
Endpoint weights	[result]	[result]	[result]	[result]
Endpoint shortcut	[result]	[result]	[result]	[result]
Free path maps	[result]	[result]	[result]	[result]
Naive combination	[result]	[result]	[result]	[result]
Encoder only ($h^{(0)}$)	[result]	[result]	[result]	[result]

two of ACM, DBLP, and IMDB, we do not claim that the intermediate nodes contribute through the instance weights. [result]

5.2 Known-Answer Tasks

Benchmark accuracy cannot show whether the instance weights learn what they are meant to learn, so we also report synthetic tasks whose answer is known. In S1′, the label of each target is carried by one decisive instance among many instances of the same schema. When the decisive instances were up-weighted by a fixed factor of 512 and only a common bias was learned, the model reached test accuracies of 0.992, 0.988, and 0.996 on three seeds, whereas equal weights stayed at chance; the operator can therefore use correct instance weights. However, the full model did not learn them: all instance weights rose together and no selection emerged. Two further tasks, S3 (instance-level selection) and S2′ (recovering a planted long schema), are run with the main protocol. [result]

5.3 Limitations

The weights scale how strongly an instance acts but cannot change its transports, which are fixed by its edges; instances are simple walks, so branching meta-structures are not represented; and the ratio of influence in the two directions of a pair is determined by the two nodes' gates. On the largest graphs, training uses local subgraphs around target nodes, which approximates the full operator outside each subgraph.

6 Conclusion

To address heterogeneous graph learning under heterophily, we proposed H2S-NN, a sheaf neural network in which path instances act inside the operator of the original heterogeneous graph. Each instance is sampled without a meta-path list, weighted from its own nodes and relations, and compared through the same edge maps that the graph uses, so that its intermediate nodes receive its message instead of disappearing into a meta-path graph. [result]

References

[Barbero *et al.*, 2022] Federico Barbero, Cristian Bodnar, Haitz Sáez de Ocáriz Borde, Michael Bronstein, Petar Veličković, and Pietro Liò. Sheaf neural networks with connection laplacians. In Alexander Cloninger, Timothy Doster, Tegan Emerson, Manohar Kaul, Ira Ktena, Henry Kvinge, Nina Miolane, Bastian Rieck, Sarah Tymochko, and Guy Wolf, editors, *Proceedings of Topological, Algebraic, and Geometric Learning Workshops 2022*, volume 196 of *Proceedings of Machine Learning Research*, pages 28–36. PMLR, 25 Feb–22 Jul 2022.

[Bodnar *et al.*, 2022] Cristian Bodnar, Francesco Di Giovanni, Benjamin Chamberlain, Pietro Lio, and Michael Bronstein. Neural sheaf diffusion: A topological perspective on heterophily and oversmoothing in gnns. *Advances in Neural Information Processing Systems*, 35:18527–18541, 2022.

[Braithwaite *et al.*, 2024] Luke Braithwaite, Alessio Borgi, Gabriele Onorato, Kristjan Tarantelli, Francesco Restuccia, Fabrizio Silvestri, and Pietro Liò. Heterogeneous sheaf neural networks. *arXiv preprint arXiv:2409.08036*, 2024.

[Chen and Song, 2025] Qin Chen and Guojie Song. Adaptive heterogeneous graph neural networks: Bridging heterophily and heterogeneity. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, pages 312–321, 2025.

[Defferrard *et al.*, 2016] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. Convolutional neural networks on graphs with fast localized spectral filtering. In *Advances in Neural Information Processing Systems*, 2016.

[Duta *et al.*, 2023] Iulia Duta, Giulia Cassarà, Fabrizio Silvestri, and Pietro Liò. Sheaf hypergraph networks. In *Advances in Neural Information Processing Systems*, 2023.

[Fiorini *et al.*, 2025] Stefano Fiorini, Hakan Aktas, Iulia Duta, Stefano Coniglio, Pietro Morerio, Alessio Del Bue, and Pietro Liò. Sheaves reloaded: A directional awakening. *arXiv preprint arXiv:2506.02842*, 2025.

[Fu and King, 2024] Xinyu Fu and Irwin King. MECCH: Metapath context convolution-based heterogeneous graph neural networks. *Neural Networks*, 170:266–275, 2024.

[Fu *et al.*, 2020] Xinyu Fu, Jiani Zhang, Ziqiao Meng, and Irwin King. Magnn: Metapath aggregated graph neural network for heterogeneous graph embedding. In *Proceedings of the web conference 2020*, pages 2331–2341, 2020.

[Guo *et al.*, 2023] Jiayan Guo, Lun Du, Wendong Bi, Qiang Fu, Xiaojun Ma, Xu Chen, Shi Han, Dongmei Zhang, and Yan Zhang. Homophily-oriented heterogeneous graph rewiring. In *Proceedings of the ACM web conference 2023*, pages 511–522, 2023.

[Hansen and Gebhart, 2020] Jakob Hansen and Thomas Gebhart. Sheaf neural networks. *arXiv preprint arXiv:2012.06333*, 2020.

[He *et al.*, 2022] Mingguo He, Zhewei Wei, and Ji-Rong Wen. Convolutional neural networks on graphs with chebyshev approximation, revisited. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[He *et al.*, 2024] Mingguo He, Zhewei Wei, Shikun Feng, Zhengjie Huang, Weibin Li, Yu Sun, and Dianhai Yu. Spectral heterogeneous graph convolutions via positive noncommutative polynomials. In *Proceedings of the ACM Web Conference 2024*, pages 685–696, 2024.

[Kim *et al.*, 2024] Jongwoo Kim, Seongyeub Chu, Hyeongmin Park, Bryan Wong, Keejun Han, and Mun Yong Yi. Leveraging multi-facet paths for heterogeneous graph representation learning. *arXiv preprint arXiv:2407.20648*, 2024.

[Kipf and Welling, 2017] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

[Li *et al.*, 2023] Juanhui Li, Harry Shomer, Haitao Mao, Shenglai Zeng, Yao Ma, Neil Shah, Jiliang Tang, and Dawei Yin. Evaluating graph neural networks for link prediction: Current pitfalls and new benchmarking. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2023.

[Li *et al.*, 2024] Chao Li, Zijie Guo, Qiuting He, and Kun He. Long-range meta-path search on large-scale heterogeneous graphs. *Advances in Neural Information Processing Systems*, 37:44240–44268, 2024.

[Li *et al.*, 2025] Jintang Li, Zheng Wei, Yuchang Zhu, Ruofan Wu, Huizhe Zhang, Liang Chen, and Zibin Zheng. Heterophily-aware representation learning on heterogeneous graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 47(9):7852–7866, 2025.

[Lin *et al.*, 2025] Junhong Lin, Xiaojie Guo, Shuaicheng Zhang, Yada Zhu, and Julian Shun. When heterophily meets heterogeneity: Challenges and a new large-scale graph benchmark. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pages 5607–5618, 2025.

[Liu *et al.*, 2014] Xiaozhong Liu, Yingying Yu, Chun Guo, and Yizhou Sun. Meta-path-based ranking with pseudo relevance feedback on heterogeneous graph for citation recommendation. In *Proceedings of the 23rd acm international conference on conference on information and knowledge management*, pages 121–130, 2014.

[Lu *et al.*, 2026] Kangkang Lu, Yanhua Yu, Ruopei Guo, Nan Cheng, Zhiyong Huang, Yunshan Ma, Meiyu Liang, Yuling Wang, Xiting Qin, Yimeng Ren, et al. Addressing graph heterogeneity and heterophily from a spectral perspective. *Neurocomputing*, page 133500, 2026.

[Lv *et al.*, 2021] Qingsong Lv, Ming Ding, Qiang Liu, Yuxiang Chen, Wenzheng Feng, Siming He, Chang Zhou, Jianguo Jiang, Yuxiao Dong, and Jie Tang. Are we really making much progress? Revisiting, benchmarking, and refining heterogeneous graph neural networks. In *Proceedings*

of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD), 2021.

[Mao *et al.*, 2023] Qiheng Mao, Zemin Liu, Chenghao Liu, and Jianling Sun. Hinormer: Representation learning on heterogeneous information networks with graph transformer. In *Proceedings of the ACM web conference 2023*, pages 599–610, 2023.

[Platonov *et al.*, 2023] Oleg Platonov, Denis Kuznedelev, Artem Babenko, and Liudmila Prokhorenkova. Characterizing graph datasets for node classification: Homophily-heterophily dichotomy and beyond. In *Advances in Neural Information Processing Systems*, 2023.

[Ribeiro *et al.*, 2026] André Ribeiro, Ana Luiza Tenorio, Juan Belieni, Amauri H Souza, and Diego Mesquita. Cooperative sheaf neural networks. In *The Fourteenth International Conference on Learning Representations*, 2026.

[Schlichtkrull *et al.*, 2018] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *The Semantic Web (ESWC)*, pages 593–607, 2018.

[Shen and Kang, 2025] Zhixiang Shen and Zhao Kang. When heterophily meets heterogeneous graphs: Latent graphs guided unsupervised representation learning. *IEEE Transactions on Neural Networks and Learning Systems*, 2025.

[Singh and Verma, 2025] Deepti Singh and Ankita Verma. An overview of heterogeneous social network analysis. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery*, 15(2):e70028, 2025.

[Sun and Han, 2013] Yizhou Sun and Jiawei Han. Mining heterogeneous information networks: a structural analysis approach. *ACM SIGKDD explorations newsletter*, 14(2):20–28, 2013.

[Sun *et al.*, 2011] Yizhou Sun, Jiawei Han, Xifeng Yan, Philip S Yu, and Tianyi Wu. Pathsim: Meta path-based top-k similarity search in heterogeneous information networks. *Proceedings of the VLDB Endowment*, 4(11):992–1003, 2011.

[Veličković *et al.*, 2018] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations (ICLR)*, 2018.

[Wang *et al.*, 2019] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S Yu. Heterogeneous graph attention network. In *The world wide web conference*, pages 2022–2032, 2019.

[Wang *et al.*, 2025] Haosen Wang, Chenglong Shi, Can Xu, Surong Yan, Rong Xie, and Pan Tang. Enhancing homophily in heterogeneous graph contrastive learning via connection strength and multi-view self-expression. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '25, page 709–718, New York, NY, USA, 2025. Association for Computing Machinery.

[Xu *et al.*, 2019] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019.

[Yang *et al.*, 2023] Xiaocheng Yang, Mingyu Yan, Shirui Pan, Xiaochun Ye, and Dongrui Fan. Simple and efficient heterogeneous graph neural network. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.

[Yang *et al.*, 2025] Xiao Yang, Xuejiao Zhao, and Zhiqi Shen. Are heterogeneous graph neural networks truly effective for node classification? A causal perspective. *arXiv preprint arXiv:2510.05750*, 2025.

[Zhu *et al.*, 2020] Jiong Zhu, Yujun Yan, Lingxiao Zhao, Mark Heimann, Leman Akoglu, and Danai Koutra. Beyond homophily in graph neural networks: Current limitations and effective designs. *Advances in neural information processing systems*, 33:7793–7804, 2020.

A Proofs

Throughout, every edge map is λ_e times an orthogonal matrix, with the same λ_e on both sides of an edge, so every transport T_v in Equation (10) is orthogonal.

Proposition 1 (Unbiased instances)**.** *For walks drawn independently, r per anchor, from $\frac{1}{2}p + \frac{1}{2}q$, $\mathbb{E}\big[\sum_f m_f L_f\big] = L_P^{\mathrm{target}}$, and each walk contributes at most $2/r$ to m_f.*

Proof. By linearity, $\mathbb{E}[\sum_f m_f L_f] = \sum_a r \sum_\omega (\frac{1}{2}p + \frac{1}{2}q)(\omega)\, \frac{1}{r}\, \frac{p(\omega)}{\frac{1}{2}p(\omega)+\frac{1}{2}q(\omega)} L_{f(\omega)} = \sum_a \sum_\omega p(\omega) L_{f(\omega)}$. The ratio $p/(\frac{1}{2}p + \frac{1}{2}q)$ is at most 2. □

Proposition 2 (Hub-node equivalence)**.** *Join a hub node z_f to every node v of f by an edge whose maps are T_v on the side of v and I on the side of the hub. Eliminating the hub state from the energy $\omega_f \sum_{v\in f} \|T_v x_v - z_f\|^2$ gives exactly the instance term of Equation* (11)*.*

Proof. For fixed x, the energy is a convex quadratic in z_f with minimizer $\frac{1}{\ell+1}\sum_v T_v x_v$; substituting it gives Equation (11), which is the Schur complement of the star Laplacian with respect to the hub. □

Proposition 3 (Bounded operator)**.** *$\Delta_0 + L_P \preceq 2D$, where D is its block diagonal; hence the spectrum of $\hat{\Delta}$ lies in $[0, 2]$ for all parameter values.*

Proof. For an edge, $\|F_{v\trianglelefteq e}x_v - F_{u\trianglelefteq e}x_u\|^2 \le 2\lambda_e^2(\|x_u\|^2 + \|x_v\|^2)$, and $\lambda_e^2 I$ is the diagonal block that the edge adds at u and at v. For an instance with $n = \ell + 1 \ge 3$ nodes and $y_v = T_v x_v$, $\sum_v \|y_v - \bar{y}\|^2 \le \sum_v \|y_v\|^2 = \sum_v \|x_v\|^2 \le 2(1 - \frac{1}{n})\sum_v \|x_v\|^2$, and $\omega_f(1 - \frac{1}{n})I$ is the diagonal block that the instance adds at each of its nodes. Summing over all terms gives the claim. □

Proposition 4 (Endpoint shortcut)**.** *If the intermediate nodes of f are eliminated from $\omega_f \sum_v \|T_v x_v - z_f\|^2$, the result is $\frac{\omega_f}{2}\|T_{v_0}x_{v_0} - T_{v_\ell}x_{v_\ell}\|^2$, an edge between the endpoints*

whose transport is the composition along the path and whose weight does not depend on any intermediate node.

Proof. With $y_v = T_v x_v$ and orthogonal T_v, the intermediate y_v are free; the minimum of $\sum_v \|y_v - \bar{y}\|^2$ over them sets every intermediate y_v to $\frac{1}{2}(y_{v_0} + y_{v_\ell})$, which leaves $\frac{1}{2}\|y_{v_0} - y_{v_\ell}\|^2$. □

Proposition 5 (Filter gain)**.** *For all parameter values, the spectrum of $A = \Gamma\hat{\Delta}B$ is real and lies in $[0, 2\max_v \gamma_v\beta_v] \subset [0, 2)$, and the filter of Equation* (13) *has gain at most $\sum_k \|W_k\|_2 \leq 1$ in the norm induced by $M = B\Gamma^{-1}$.*

Proof. A is similar to $(\Gamma B)^{1/2}\hat{\Delta}(\Gamma B)^{1/2}$, which is symmetric, positive semidefinite, and bounded by $2\max_v \gamma_v\beta_v$; moreover $MA = B\hat{\Delta}B$ is symmetric, so A is self-adjoint in $\langle\cdot, M\cdot\rangle$. Hence the spectrum of $A - I$ lies in $[-1, 1)$ and $\|C_k(A - I)\|_M \leq 1$. Because M acts on each node by a scalar, it commutes with the per-node map W_k, whose M-norm is $\|W_k\|_2$. □

B Prediction Heads

For node classification, $\hat{y}_v = \mathrm{MLP}([x_v \parallel h_v^{(0)}])$. For link prediction, both endpoints use $z_v = [x_v \| h_v^{(0)}]$ and a relation-specific bilinear decoder scores the pair. For graph classification, node states are summed over the graph and passed to an MLP; sum pooling is permutation invariant and can distinguish multisets [Xu *et al.*, 2019].

C Datasets, Settings, and Full Results

Dataset statistics (nodes, facts, node and relation types, labelled targets, $\mathcal{H}^2$ and MLH per dataset and per genre for IMDB), the tuning budget and search spaces, the WISP settings per dataset, link-prediction results with Hits@10, AUROC and AUPR and the seen/cold-start split, and graph classification on MUTAG, PROTEINS, ENZYMES, and NCI1 (atom and bond labels read as node and relation types) are reported here. [result]